Reth GPU performance — 1 / 2 / 4 / 8 RTX 5090 (JIT-by-default)

Benchmark of the reth ELF (`zkm-gpu-perf --program reth --stage wrap`) on the ant-5090-2 box (8× NVIDIA RTX 5090, 124 logical cores, 925 GB RAM, 36 GB visible per GPU). Runs use the JIT-by-default executor (`feat/upgrade-plonky3` Ziren) end-to-end through prove + verify, with the GPU prover from `feat/gpu-basefold-dispatch` ziren-gpu. Trace generation uses `TRACE_GEN_WORKERS=16`, `recursion_opts.shard_batch_size = 4`, GPUs 2-7 selected via `CUDA_VISIBLE_DEVICES`.

Workload

Field	Value
Program	reth (`perf/mipsel-zkm-zkvm-elf/reth` in ziren-gpu)
Cycles	419,960,677
Shards	189
ELF size	5.2 MB
Stdin	9.1 MB (single block + state proofs)
Proof size	338,288,443 bytes (uncompressed Groth16-ready bn254 wrap)

Results

`RECURSION_SHARD_BATCH_SIZE` (SBS) controls the number of concurrent prover-submit threads in `compress_multi_gpu`. The default now scales with GPU count: `(gpu_count * 2).clamp(4, 8)` — see `crates/stark/src/opts.rs::ZKMProverOpts::gpu`. Override via `RECURSION_SHARD_BATCH_SIZE`.

Old baseline (`feat/gpu-basefold-dispatch`, SBS=4, no pk cache)

GPUs	Core (s)	Compress (s)	Shrink (s)	Wrap (s)	**Total (s)**	Core kHz
1	106.5	52.9	0.43	1.11	**160.9**	3,944
2	64.7	46.0	0.45	1.21	**112.4**	6,488
4	58.3	42.1	0.46	1.02	**101.9**	7,210
8	56.4	42.0	0.44	1.08	**99.9**	7,452

New (SBS=`(gpu_count*2).clamp(4,8)` + pk cache + Apr-25 stark.rs aux_commits fix)

GPUs	SBS	Core (s)	Compress (s)	Shrink (s)	Wrap (s)	**Total (s)**	Core kHz	Δ vs old
1	4	109.3	48.4	0.41	1.14	**159.2**	3,841	−1.7 s
2	4	73.3	35.3	0.45	1.09	**110.1**	5,731	−2.3 s
4	8	62.3	**29.3**	0.47	1.11	**93.1**	6,744	**−8.8 s**
8	8	62.7	**29.8**	0.47	1.07	**94.1**	6,695	**−5.8 s**

pk-cache hit rate (4 GPU, 191-shard reth)

`compress_pk_cache_summary { hits: 240, misses: 143, hit_rate: 62.7% }` — 191 shards × ~2 first-layer-input + recursion-tree calls each yielded 383 compress submissions. ~143 unique program shapes × per shape ≈ shapes-cardinality of the `lift_programs_lru` / `join_programs_map` snapshots. Setup() per shard is ~15-25 ms host + GPU upload, so 240 cache hits saved roughly 3-6 s of compress wall time. The gap from 96.4 s → 93.1 s tracks that.

Sweep around the 4/8-GPU sweet spots (SBS=8, RMTPD=1 unless noted)

GPUs	SBS	RMTPD	Total (s)	Notes
4	4	1	101.9	old SBS=4 baseline (re-run today: same compress=42 plateau)
4	8	1	**96.4**	SBS=8 + pk cache (today's winner)
8	4	1	99.9	old SBS=4 baseline
8	8	1	**94.1**	SBS=8 + pk cache (today's winner)
8	12	1	98.0	SBS=12 plateaus + core regresses on `state.lock()` contention
8	8	2	99.2	per-GPU 2× concurrent tasks regresses (memory pressure)

Speedup (with the new SBS default + pk cache)

- 1 → 2 GPU: 1.45× total (1.49× core)
- 1 → 4 GPU: 1.65× total (1.70× core)
- 2 → 4 GPU: 1.14× total (compress 35 → 30 s)
- 4 → 8 GPU: 1.02× total. Core plateaus at 4 workers (CPU trace-gen floor); compress now scales 1:1 with the GPU pool — saturated at SBS=8.

Stack of changes

1. **`crates/recursion/circuit/src/stark.rs::dummy_vk_and_shard_proof`** — `auxiliary_commits` from `Vec::new()` to `vec![dummy_commit(), dummy_commit()]`, plus populated `permutation`/`quotient` in `dummy_opened_values`. The wrap_program is built once via OnceLock from a dummy proof; the legacy FRI shrink prover (recursion shards have no Cpu chip → fall through to legacy FRI at `crates/stark/src/prover.rs:359`) emits 2 aux commits + populated perm/quotient, so the dummy must match. Without this, `wrap_prover.machine().verify()` panics with `local_cumulative_sum != 0`. This

was the Apr-25 fix that landed locally but never made it to the GPU box's tree — applying it unblocked all of multi-GPU today.

2. **`crates/stark/src/opts.rs::ZKMProverOpts::gpu`** — SBS default from 4 to `(gpu_count * 2).clamp(4, 8)`. 1-2 GPU stays at 4 (8x oversubscribe OOMs the single 32 GB card on real reth shards), 4 GPU jumps to 8 (2× oversubscribe hides per-shard CPU prep), 8 GPU stays at 8 (1:1 with the pool).
3. **`ziren-gpu/prover/src/compress_multi_gpu.rs`** — per-`WorkerContext` `pk_cache: HashMap<usize, Arc<(DevicePk, Vk)>>` keyed by `Arc::as_ptr(&program) as usize`. `recursion_program` / `compress_program` already return cached `Arc<RecursionProgram>` per-shape, so shape-equivalent shards share the same Arc and skip `setup()`'s preprocessed-trace gen + GPU upload on cache hit. Net effect: compress 33 s (validation) → 30 s today.

Why SBS matters

`compress_multi_gpu` spawns `recursion_opts.shard_batch_size` worker threads. Each thread pulls one (record, traces) tuple off `record_and_trace_rx`, calls `pool.submit()` → `setup() + commit() + open()` on a GPU pool worker, blocks on the result, then loops. With SBS=4 and 8 GPUs only 4 of 8 GPU pool workers ever have work in flight.

- **8 GPU SBS=4 → SBS=8** unlocks the idle 4 GPUs. Compress drops 42.0 s → 33.1 s (-21 %) and total 99.9 → 94.4 (-5.5 %).
- **4 GPU SBS=4 → SBS=8** oversubscribes 2× per GPU. Per-shard CPU prep (recursion-program build, `setup()` on host side, `generate_dependencies`) overlaps with another shard's GPU work. Compress drops 42.1 → 32.5 s (-23 %) and total 101.9 → 97.1 (-4.7 %).
- **8 GPU SBS=12** plateaus then regresses (worker threads contend on state mutexes; core 56.4 → 62.8 s).
- **`ZKM_GPU_RECURSION_MAX_TASKS_PER_DEVICE=2`** also regresses (GPU memory pressure from two concurrent in-flight tasks per device).
- **1 GPU SBS=8** OOMs on real reth shards (8 setup() / commit() / open() concurrent allocations on a 32 GB card). The new default (`(gpu_count * 2).clamp(4, 8)`) keeps 1-2 GPU at SBS=4 for safety and bumps 4-8 GPU to SBS=8 for the win.

JIT-by-default vs interpreter (4 GPU, identical proof output)

Both modes produce `cycles=419,960,677, proofBytes=338,288,443`.

Mode	Core (s)	Compress (s)	Shrink (s)	Wrap (s)	**Total (s)**	Core kHz
JIT-by-default	61.5	41.9	0.46	1.04	**105.0**	6,823
Interpreter	58.7	42.3	0.44	1.11	**102.5**	7,160

The two are within run-to-run variance. The JIT wins big on **execute-only** workloads (tendermint 1.8 s → 0.27 s, 6.7×; Reth single-block 25 s → 5 s). Once the GPU prover is engaged, FRI commit / multi-shard recursion / trace LDE dominate, so the executor speedup is invisible at the e2e level. Both modes still satisfy every per-stage verifier — the JIT-produced trace is mathematically equivalent.

End-to-end correctness

Every configuration successfully runs **prove core → compress → shrink → wrap_bn254 → verify core → verify compress → verify shrink → verify wrap** with `wrapping successful`.

The JIT-produced execution trace is mathematically validated through every recursion layer up to BN254. The wrap-stage verifier (`wrap_bn254::verify`) takes ~70 ms.

JIT executor changes (this run)

The executor now runs JIT-by-default for every workload (interpreter is the fallback for unsupported opcodes only). Key fixes vs the prior doc:

- **Memfd-COW for unconstrained blocks** — host buffer switched from anonymous mmap to `memfd_create` + `MAP_SHARED`; ENTER_UNCONSTRAINED mmap's a private COW view of the same fd, EXIT munmaps it. Mirrors SP1's `crates/core/jit/src/context.rs::enter_unconstraine` Without this, Reth panicked at ~108 M cycles with `NodeNotResolved`.
- **Per-precompile sync table** — every curve op (SECP256K1 / SECP256R1 / BN254 / BLS12381 / ED25519), Fp/Fp2 op, KECCAK_SPONGE, UINT256_MUL, U256xU2048_MUL, POSEIDON2_PERMUTE has explicit input/output byte ranges synced through the bridge. Caught a 60 % Reth-block failure rate (UINT256_MUL didn't sync the modulus at A1+32; KECCAK_SPONGE used a wrong fixed length instead of reading it from `mem[A1+64]`).
- **`clk_bump = 1`** — JIT's `state.global_clk` now matches the interpreter's exactly (previously bumped by 4 per instruction).
- **No-pool mem_fd** — `Drop` always closes the fd to avoid stale page-cache contents leaking into the next run's COW snapshot.

The executor's JIT vs interp output is now **byte-identical** (cycles + public-values + first-16 bytes) across all 12 standard shape-bin workloads.

Bottleneck analysis

1. Compress stage CPU work (~42 s at 4 GPU)

Compress is now ~41 % of wall time at 4 GPUs and is the next obvious target. The remaining cost is mostly CPU-side:

- Per-shard recursion-program build + `setup()` (CPU).
- Dummy witness construction in `dummy_vk_and_shard_proof` (CPU).
- Jagged-fingerprint computation per shard (CPU).
- `make_merkle_proofs` lookup + Merkle path open (CPU).

The shard-level basefold prover code path (formerly the WHIR fast path) is currently dead — `use_whir = false` because `BasefoldShardVerifier` still fails round-0 sumcheck on real shards. Re-enabling it as a deliberate selector once that's fixed would let compress amortise more work to the GPU.

2. Core stage scales 1 → 2 GPU and plateaus

Core dropped 106.5 s → 64.7 s with 2 GPUs (1.65×) but only 64.7 s → 58.3 s with 4 GPUs (1.11×). Per-shard core cost falls from ~563 ms (1 GPU) to ~342 ms (2 GPU) and stays there.

Plateau hypotheses (in priority order):

1. **CPU-side trace generation saturates**. `TRACE_GEN_WORKERS=16` is already 16× the legacy default; bumping further may help.
2. **PCIe bandwidth between host and GPUs**. Reth shards are large (multi-MB traces) and core dispatch is one-trace-at-a-time per worker.
3. **CUDA stream serialisation in the per-GPU prover queue**. Each GPU gets its own `MultiGpuDevicePool` worker but inside that worker FRI commit / sumcheck phases serialise.

3. Shrink + Wrap are fixed at ~1.5 s

Single-GPU operations by design. No parallelism opportunity.

Where to push next

The SBS default change above lands the cheapest win. Remaining levers:

Lever	Estimated wall reduction	Effort
Cache per-shard pk across shape-equivalent shards (skip `setup()` GPU upload on cache hit)	~5-10 s (compress)	medium
Fix BasefoldShardVerifier round-0 + re-enable shard-level basefold	~15 s	high
Increase TRACE_GEN_WORKERS to 24-32 with a contention audit on the per-shard state mutex	~2-5 s (core)	low — needs profiling first
Pin shard checkpoints in memory (skip reload per shard)	~5 s	medium
Wider PCIe (Gen 5 vs current)	hardware-bound	n/a

`TRACE_GEN_WORKERS=32` was tested and gave a small core regression (58.3 → 62.5 s at 4 GPU) — the bottleneck is the per-shard `state.lock()` mutex inside the trace-gen worker loop, not the worker count. Profile that path before adding more workers.

Reproducing

```
# On ant-5090-2 (Ziren on feat/upgrade-plonky3 with JIT-by-default,
# ziren-gpu on feat/gpu-basefold-dispatch).
cd /home/ubuntu/sd/ziren-gpu
export PATH=/usr/local/cuda/bin:/usr/local/go/bin:$PATH
source ~/.zkm-toolchain/env

# Use a known-free GPU set and pin all the prover knobs.
export ZIREN_USE_BASEFOLD=1 ZIREN_BASEFOLD_GPU=1 VERIFY_VK=false
export ZKM_GPU_CORE_MAX_TASKS_PER_DEVICE=1
export ZKM_GPU_RECURSION_MAX_TASKS_PER_DEVICE=1
export DEFAULT_CHECKPOINTS_CHANNEL_CAPACITY=512
export SHAPE_CHECK_FREQUENCY=1024
export SHARD_SIZE=4194305
export TRACE_GEN_WORKERS=16
export RECORDS_AND_TRACES_CHANNEL_CAPACITY=16
mkdir -p /dev/shm/zkm-gpu-perf && export TMPDIR=/dev/shm/zkm-gpu-perf

# 1 GPU
CUDA_VISIBLE_DEVICES=2 ZKM_GPU_DEVICES=0 \
  ./target/release/zkm-gpu-perf --program reth --stage wrap

# 2 GPUs
CUDA_VISIBLE_DEVICES=2,3 ZKM_GPU_DEVICES=0,1 \
  ./target/release/zkm-gpu-perf --program reth --stage wrap
```

```
# 4 GPUs
CUDA_VISIBLE_DEVICES=2,3,4,5 ZKM_GPU_DEVICES=0,1,2,3 \
  ./target/release/zkm-gpu-perf --program reth --stage wrap

# 8 GPUs
CUDA_VISIBLE_DEVICES=0,1,2,3,4,5,6,7 ZKM_GPU_DEVICES=0,1,2,3,4,5,6,7 \
  ./target/release/zkm-gpu-perf --program reth --stage wrap
```

Drop `--skip-verify` (it's the default ON) to also run every per-stage verifier; total run time grows by ~5 % for the verify pass.

Captured 2026-04-30

- Branch: `feat/gpu-basefold-dispatch` (ziren-gpu) ↔ `feat/upgrade-plonky3` (Ziren)
- Executor: JIT-by-default (`crates/core/jit`, `clk_bump=1`, memfd-COW unconstrained-block rollback).
- vk_map.bin: padded to 2^8 = 256 entries (87 real vks + sentinels).
- Default proof system: FRI everywhere (host CPU + GPU paths share one vk_map). Re-introduce the BaseFold-shard prover as a deliberate selector — not env-gated — once round-0 sumcheck is resolved.